

PART VI — THE SELF-LEARNING LAYER

Chapter 36

Evaluating Whether Self-Learning Actually Works

“A growing memory collection is evidence of activity. Only a repeated, comparable measurement is evidence of improvement.”

Chapter 36 — Evaluating Whether Self-Learning Actually Works

This chapter builds the evaluation discipline that answers that question — a fixed baseline, a repeatable fixture-driven test harness, and a comparison methodology precise enough to catch a regression that an aggregate score would hide. The chapter leans heavily on `run_batch.py`, a tool already introduced in earlier chapters as a source of real, grounded examples (the ASD-disambiguation batch in Chapter 31.7, the self-learning pipeline batch in Chapter 35.3) but never examined for what it actually is: a genuinely clever piece of test infrastructure that drives the unmodified, production `agent_query.main()` entry point through a scripted conversation, with no changes to the code under test at all.

36.1 Baseline accuracy before learning kicks in

Definition — Baseline run

A recorded set of answers to a fixed question set, captured before any self-learning mechanism has had a chance to influence retrieval — the reference point every later comparison measures against.

A claim like "the learned `qa` collection improved answer quality" is unfalsifiable without a baseline captured *before* that collection had any entries to retrieve from. This is not a subtle methodological point — it is the entire difference between a genuine before/after comparison and a single after-only snapshot that merely describes the system's current state with no reference point to measure improvement against.

Capturing a baseline is operationally simple given Chapter 35's `stats` command: run the fixed evaluation set (Section 36.2) against a fresh deployment with `learned_qa` at zero entries, archive every answer alongside its `request_id`, and only then let the system accumulate real usage and distilled memory. Skipping this step is the single most common way an evaluation effort fails before it starts — by the time someone thinks to ask "did this help," the collection has already grown past zero and the true starting point is gone for good.

A subtler trap is capturing a baseline that is technically "before learning" but not actually comparable to the run it will later be measured against. If the source corpus changes between baseline and comparison — new documents ingested, old ones removed — any observed difference conflates corpus drift with learning effect, and the evaluation can no longer isolate which one actually produced a given change in answer quality. A rigorous baseline freezes not just `learned_qa` at zero, but the entire evaluation context: same corpus, same model, same configuration flags (Chapter 36B), varying only the one thing under test.

36.2 Building a fixed evaluation set

`run_batch.py`'s fifteen `BATCHES` entries are not fifteen independent smoke tests — read together, they are a single, deliberately structured 100-question fixed evaluation set, each batch targeting a specific failure category: disambiguation (Batch 1), zero-chunk situations (Batch 3), multi-source synthesis (Batch 4), hallucination-risk partial-knowledge-base topics (Batch 5), and — directly relevant to this chapter — the self-learning pipeline itself (Batch 12).

Batch	Category	What it stresses
1	ASD disambiguation + thumbdown	Cross-domain acronym collision, Chapter 31's mechanism
3	Zero-chunk situations	Out-of-corpus queries, Chapter 30's blocklist
5	Hallucination-risk partial KB	Grounding discipline under incomplete evidence
12	Self-learning pipeline	stats → learn → stats → re-ask, this chapter's exact subject
13	Multi-turn thumbdown refinement	Whether feedback steers the next retrieval

A fixed evaluation set earns the adjective "fixed" by staying byte-for-byte identical across every run it is used for — the moment a question's wording changes between one evaluation and the next, any difference in the answer conflates two variables (did the system change, or did the question) that a genuine before/after comparison needs kept separate.

36.3 Driving repeated runs with `run_batch.py`

`run_batch.py` solves the repeatability problem with a technique worth understanding in its own right: `_ScriptedStdin` replaces `sys.stdin`, and `builtins.input` is monkey-patched to pull from a scripted list of questions and commands instead of a human typing at a keyboard — then `agent_query.main()` is imported and called completely unmodified. The production code has no idea it is being tested; every `input("Your question: ")` call inside it receives the next scripted entry exactly as if a person had typed it.

Figure 36.2 traces this substitution at exactly the layer it happens — not inside `agent_query.py` itself, which remains completely untouched, but one level below it, at the boundary Python's own `input()` builtin crosses to reach the terminal.

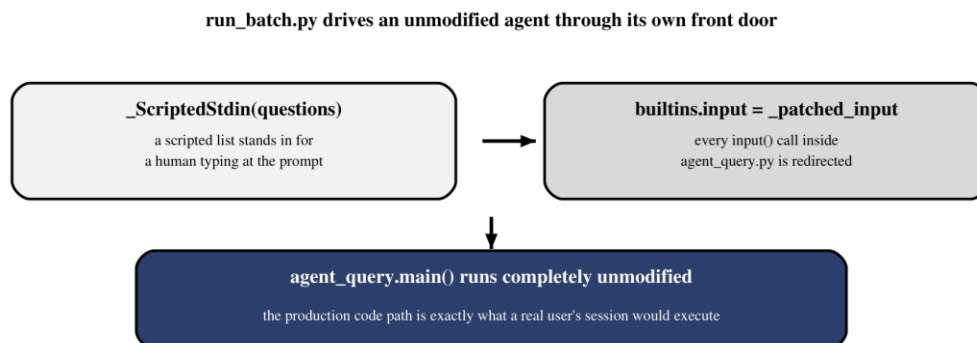


Figure 36.2 — Monkey-patching `stdin`, not the agent itself, means the exact production code path runs under test — no test-only branches, no mocked internals.

```
def _patched_input(prompt="") :
    sys.stdout.write(prompt)
    sys.stdout.flush()
    return scripted.readline().rstrip("\n")

import builtins
builtins.input = _patched_input
...
import agent_query
agent_query.main()
```

This is a meaningfully stronger testing discipline than calling `run_agent()` directly with hand-constructed arguments would be — it exercises the *entire* interactive loop, including the REPL prompt logic, the `'bad'/'stats'/'learn'` command dispatch Chapter 35 covered, and the exact sequencing a real session produces, all without a single line of `agent_query.py` needing to know it is under test. A `'_Tee'` class simultaneously mirrors every line of output to both the live terminal and a timestamped log file, so a batch run produces both real-time visibility and a permanent, diffable record.

36.4 Measuring answer quality over time

Chapter 16.3's automatic `check_answer_quality()` verdict — `'OK'` or `'INSUFFICIENT'` — is one legitimate signal to track across repeated baseline runs, and it costs nothing extra to collect since it already runs on every interaction. But it answers only "was this answer grounded," not "was this the *best* answer the system could have given" — a subtler question that generally still needs human judgment, or an LLM-as-judge comparison against the archived baseline answer for the identical question.

The same fixed eval set, run at two points in time

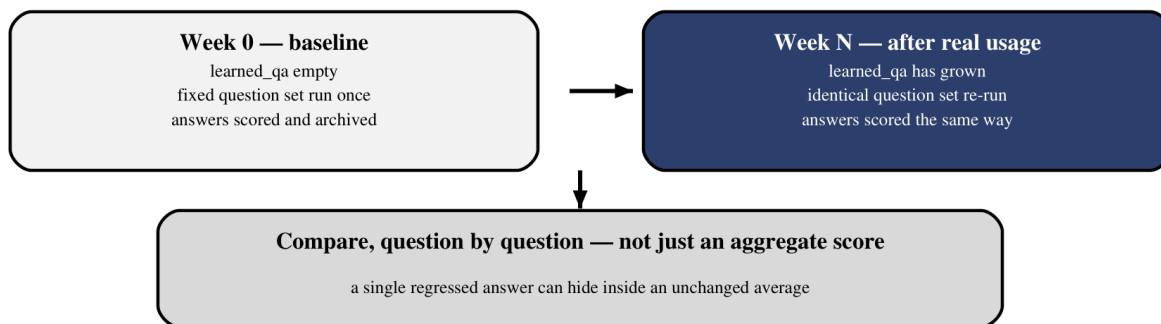


Figure 36.1 — The same fixed question set, scored the same way, run at two separated points in time, compared question by question rather than only as an aggregate.

The comparison's granularity matters as much as the metric itself. An aggregate "quality score" that stays flat between baseline and a later run can hide real movement in both directions — three questions genuinely improved by newly-distilled memory, three questions genuinely regressed by something else entirely, netting out to no visible change at all. Comparing question-by-question, not only in aggregate, is the only way to catch that kind of cancellation.

This granularity requirement is exactly why Figure 36.1 draws the comparison as two archived, timestamped snapshots rather than a single trend line — a trend line invites reading only its slope, while

two explicit snapshots, kept and compared question by question, invite the more demanding but more honest question of which specific answers moved, and in which direction.

36.5 Detecting memory drift and regression

"Memory drift" names a specific risk Chapter 28.5 already previewed in principle: as `learned_qa` grows, its influence on retrieval grows with it (Chapter 33.5), and if any distilled entry was subtly wrong — grounded in a source that was itself outdated, or a rephrasing that drifted slightly from its original meaning during Chapter 32's distillation step — that entry's influence compounds with every future query it happens to match, rather than staying an isolated, one-time error.

Detecting drift requires exactly the repeated-baseline discipline Sections 36.1 and 36.4 already built: a question the fixed evaluation set answered correctly at week 0 that now answers *worse* at week 8, despite the corpus itself being unchanged, is the signature symptom. This is precisely why the evaluation set must stay fixed and the comparison must stay question-by-question — a regression on one specific, previously-correct question is exactly the kind of local signal an aggregate score is most likely to bury.

Common pitfall — Treating a growing `learned_qa` count as a health metric on its own

Chapter 35.2's `stats` command reports collection size, not collection correctness. A steadily growing count with no accompanying baseline comparison is consistent with both a genuinely improving system and a slowly drifting one — the count alone cannot distinguish between them.

36.6 When to manually review and prune the learned collection

Section 36.5's drift detection tells you *that* a regression happened; it does not by itself tell you *which* `learned_qa` entry caused it, since a single query typically matches against several stored entries at once. A manual review pass — reading through the `original_query`, `question`, and `answer` metadata Chapter 32.6 stores per entry — is the practical next step once drift is suspected on a specific question, narrowing the search to whichever entries plausibly matched that question's embedding.

Chapter 35.4's hypothetical `/forget` command is the tool this workflow is missing today — this project's shipped design has no CLI path for removing a specific entry once manual review identifies it as the culprit, only direct ChromaDB collection access outside any command this book has covered. A practical periodic review cadence — monthly, or triggered specifically by a detected regression — is a reasonable operational default even without that tooling gap closed, since a slow, curated pruning process still beats an uncurated collection that only ever grows.

36.7 Comparing to research benchmarks

This project's own evaluation approach is worth placing next to the broader published literature on evaluating retrieval-augmented and self-improving systems, not to claim parity with any of them, but to understand what each measures and where this chapter's methodology sits relative to that landscape. RAGAS, a widely used open-source evaluation framework, decomposes RAG quality into separable metrics — faithfulness (are claims grounded in retrieved context), answer relevancy, context precision, and context recall — computed largely via LLM-as-judge scoring rather than exact-match comparison. Self-RAG, a research architecture rather than an evaluation tool, trains a model to emit explicit reflection tokens deciding when to retrieve and critiquing its own generation's support and relevance, treating quality control as something learned into the model rather than layered on top of it as a separate pipeline stage. Pistis-RAG frames its contribution around trustworthiness specifically — content-centric, multi-stage scaffolding aimed at keeping a RAG pipeline's outputs verifiably grounded across a longer generation process.

Framework	What it measures	Relation to this project's approach
RAGAS	Faithfulness, relevancy, context precision/recall — via LLM judge	Conceptually close to <code>check_answer_quality</code> (Ch16) and <code>validate_retrieval</code> (Ch11)
Self-RAG	Learned reflection tokens deciding retrieve/critique	This project's judge logic is a separate pipeline stage, not learned into the model
Pistis-RAG	End-to-end trustworthiness across multi-stage generation	Analogous in spirit to the NAC→DC→LBC faithfulness guards (Ch22B)

The honest comparison to draw is architectural, not a leaderboard ranking: this project's `check_answer_quality()`, `validate_retrieval()`, and the compression-stage faithfulness guards are all separate, auditable, swappable pipeline stages rather than behavior trained into the model itself — closer in philosophy to RAGAS's decomposed, externally-measured metrics than to Self-RAG's internalized reflection. This project's fixed-batch evaluation methodology (Sections 36.1-36.5) is a lighter-weight, project-specific instrument than any of these published frameworks — genuinely useful for catching this specific deployment's own regressions, but not a substitute for the broader, cross-system benchmarking these frameworks are actually designed to support.

36.8 The long-lived single-process benchmark runner

`run_batch.py` launches a fresh Python process per invocation, appropriate for `app/`'s imperative architecture but a poor fit once `app_workflow/`'s LangGraph rewrite (Chapter 19B) introduced genuinely stateful, expensive-to-initialize services — the MongoDB client, tracing setup (Phoenix/LangSmith/Langfuse), the learned-QA vector-store view — that a fresh process re-pays the cost of on every single batch. This project's own `run_all_workflow_batches.py`, documented in `Architecture.md`'s 2026-07-13 and 2026-07-14 entries, addresses exactly this: it starts one long-lived `app_workflow/api.py` subprocess, polls `GET /stats` until the service reports ready, and then fires all fifteen `BATCHES` — the identical fixture dictionary Section 36.2 already covers, reused rather than reimplemented — as roughly 100 HTTP requests against that one persistent process, with no inter-query delay.

The architectural payoff named in this project's own history is state persistence across the entire suite: in-memory services, tracing context, the MongoDB client connection, and the learned-QA collection's live view all survive from the first scripted question to the last, meaning Batch 12's self-learning pipeline test (ask, force-learn, verify count grew, re-ask) and every batch after it observe the **same**, continuously-evolving `learned_qa` state — a far closer approximation of a real multi-user deployment's actual operating conditions than fifteen independently-launched, freshly-initialized processes would provide.

This design also makes drift detection (Section 36.5) meaningfully cheaper to run at scale: a single long-lived process working through a hundred scripted scenarios back-to-back produces one continuous, chronologically ordered log rather than fifteen disjoint ones that would need to be stitched together afterward just to reconstruct the order in which the learned collection actually evolved during the run.

Part VI closes with this chapter's core lesson stated plainly: every mechanism from Chapter 29's feedback ledger through Chapter 35's CLI commands is only as trustworthy as the evaluation discipline that can detect when one of them stops working. Part VII turns from the self-learning layer specifically to the broader concerns of running this whole system in production — deployment, observability at scale, and the operational discipline the rest of this book's mechanisms all eventually depend on.